

SQL Query Results

Question 1

Question

How many entries are from applicants who applied for Fall 2026?

Result

Fall 2026 applicant count: 33,207

SQL

```
SELECT COUNT(*)  
FROM applicants  
WHERE term ILIKE 'fall 2026';
```

Explanation

This counts every row where the term is Fall 2026. I used ILIKE so it doesn't matter how the term is capitalized based on possible discrepancies in the entries.

Question 2

Question

Among entries that give a nationality, what percentage are international students?

Result

Percent international: 47.14%

SQL

```
SELECT ROUND(  
    100.0 * COUNT(*) FILTER (WHERE us_or_international ILIKE 'international')  
    / COUNT(*), 2)  
FROM applicants  
WHERE us_or_international IS NOT NULL  
    AND TRIM(us_or_international) <> '';
```

Explanation

The WHERE line keeps only rows with a real nationality, so NULL and blank values are left out of the bottom number. American and Other stay in the bottom number, they just don't count as international. FILTER counts only the international rows for the top number, then I divide and multiply by 100 and round to 2 decimals.

Question 3

Question

What are the average GPA, GRE Quantitative, GRE Verbal and GRE Analytical Writing scores of applicants who provide each metric?

Result

Average GPA: 3.76

Average GRE Quantitative: 165.68

Average GRE Verbal: 160.35

Average GRE Analytical Writing: 4.32

SQL

```
SELECT ROUND(AVG(gpa)::numeric, 2),  
       ROUND(AVG(gre)::numeric, 2),  
       ROUND(AVG(gre_v)::numeric, 2),  
       ROUND(AVG(gre_aw)::numeric, 2)  
FROM applicants;
```

Explanation

There are four separate AVG calculations in one query. AVG skips NULL values by itself, so each average only uses the applicants who actually gave that score. When I loaded the data I turned zeros and out-of-range scores into NULL, so they don't get averaged in. This I believe would make the result for average even more accurate based on the sample gotten.

Question 4

Question

What is the average GPA of American applicants who applied for Fall 2026?

Result

Average GPA: 3.79

SQL

```
SELECT ROUND(AVG(gpa)::numeric, 2)  
FROM applicants  
WHERE term ILIKE 'fall 2026'  
       AND us_or_international ILIKE 'american'  
       AND gpa IS NOT NULL;
```

Explanation

The WHERE line keeps only the Fall 2026 rows, classified as American, that have a GPA. Then AVG takes the average of those GPAs.

Question 5

Question

What percentage of Fall 2025 entries are acceptances?

Result

Fall 2025 acceptance percentage: 39.04%

SQL

```
SELECT ROUND(  
    100.0 * COUNT(*) FILTER (WHERE status ILIKE 'accepted') / COUNT(*), 2)  
FROM applicants  
WHERE term ILIKE 'fall 2025';
```

Explanation

The WHERE line here keeps only Fall 2025 rows, so COUNT(*) is the bottom number (all Fall 2025 entries). FILTER counts only the rows with status accepted for the top number. Dividing gives the share that were accepted.

Question 6

Question

What is the average GPA of accepted applicants who applied for Fall 2026?

Result

Average GPA: 3.76

SQL

```
SELECT ROUND(AVG(gpa)::numeric, 2)  
FROM applicants  
WHERE term ILIKE 'fall 2026'  
    AND status ILIKE 'accepted'  
    AND gpa IS NOT NULL;
```

Explanation

I maintained the same idea as Question 4 here, but the filter is status accepted instead of American. It keeps Fall 2026, accepted rows that have a GPA and averages them.

Question 7

Question

How many entries are from applicants who applied to Johns Hopkins University for a master's degree in Computer Science?

Result

Johns Hopkins master's Computer Science entries: 20

SQL

```
SELECT COUNT(*)  
FROM applicants  
WHERE program ILIKE '%computer science%'  
    AND (program ILIKE '%johns hopkins%'  
        OR program ILIKE '%john hopkins%'  
        OR program ILIKE '%jhu%')  
    AND degree ILIKE 'master%';
```

Explanation

This uses the original program and degree fields. The program has to contain the phrase computer science (just "computer" would also catch Electrical And Computer Engineering). It also has to contain Johns Hopkins, the misspelling John Hopkins, or JHU, and the degree has to start with master.

Question 8

Question

How many Fall 2026 entries are acceptances for a PhD in Computer Science at Georgetown, MIT, Stanford or Carnegie Mellon (using the original fields)?

Result

Original-field count: 30

SQL

```
SELECT COUNT(*)
FROM applicants
WHERE term ILIKE 'fall 2026'
      AND status ILIKE 'accepted'
      AND degree ILIKE 'phd'
      AND program ILIKE '%computer science%'
      AND (program ILIKE '%georgetown%'
           OR program ILIKE '%massachusetts institute of technology%'
           OR program ~* '\mmit\M'
           OR program ILIKE '%stanford%'
           OR program ILIKE '%carnegie mellon%');
```

Explanation

Every condition has to be true at once, so they are joined with AND: Fall 2026, accepted, PhD, and computer science in the program. The four universities are grouped in parentheses with OR, since a row only needs to match one of them. MIT is matched either by the full name or by the standalone word MIT (the regex `\mMIT\M`), so it doesn't match words like Smith.

Question 9

Question

Repeat Question 8, but find the university and program using the LLM-generated fields.

Result

Original-field count: 30

LLM-field count: 30

Difference: +0

SQL

```
SELECT COUNT(*)
FROM applicants
WHERE term ILIKE 'fall 2026'
      AND status ILIKE 'accepted'
```

```

AND degree ILIKE 'phd'
AND llm_generated_program ILIKE '%computer science%'
AND (llm_generated_university ILIKE '%georgetown%'
     OR llm_generated_university ILIKE '%massachusetts institute of technology%'
     OR llm_generated_university ~* '\mmit\M'
     OR llm_generated_university ILIKE '%stanford%'
     OR llm_generated_university ILIKE '%carnegie mellon%');

```

Explanation

This is the same query as Question 8, but the program and university checks use llm_generated_program and llm_generated_university. Term, degree and status still use the original fields. The counts came out the same, and I checked that it was the same 30 entries and not just a matching number. I think it's because these four universities are usually written pretty consistently, so the LLM cleanup didn't change which rows match. It could make a bigger difference for smaller schools where people use abbreviations or typos.

Question 10

Question

Do international applicants report a higher average GRE Quantitative score than American applicants?

Result

American: 163.60 (370 applicants)

International: 166.63 (800 applicants)

SQL

```

SELECT us_or_international,
       COUNT(gre),
       ROUND(AVG(gre)::numeric, 2)
FROM applicants
WHERE (us_or_international ILIKE 'american'
      OR us_or_international ILIKE 'international')
      AND gre IS NOT NULL
GROUP BY us_or_international
ORDER BY us_or_international;

```

Explanation

This groups the rows by nationality with GROUP BY and finds the average GRE Quantitative for each group. I only kept American and International rows with a GRE Quantitative score, and COUNT(gre) shows how many applicants each average is based on. International applicants report a higher average, but only about 1,170 people gave a valid score, so I wouldn't read too much into a gap of 3 points.

Question 11

Question

For Fall 2026, is the average GPA of accepted applicants higher than the average GPA of rejected applicants?

Result

Accepted: 3.76 (7,911 applicants)

Rejected: 3.78 (8,794 applicants)

SQL

```
SELECT status,
       COUNT(gpa),
       ROUND(AVG(gpa)::numeric, 2)
FROM applicants
WHERE term ILIKE 'fall 2026'
      AND (status ILIKE 'accepted' OR status ILIKE 'rejected')
      AND gpa IS NOT NULL
GROUP BY status
ORDER BY status;
```

Explanation

This keeps Fall 2026 rows with a GPA that are either accepted or rejected, groups them by status and averages the GPA for each. An interesting thing here is that I expected accepted applicants to have the higher GPA, but the two are almost the same, and rejected is slightly higher. That suggests GPA alone doesn't separate the two groups much in this data, and since GPAs are self-reported, one thing to have in mind is that people may have also rounded up.